



The Definitive Guide to Firebase

Build Android Apps on Google's Mobile
Platform

—
Laurence Moroney

Apress®

The Definitive Guide to Firebase

Build Android Apps on Google's Mobile Platform



Laurence Moroney

Apress®

The Definitive Guide to Firebase: Build Android Apps on Google's Mobile Platform

Laurence Moroney
Seattle, Washington, USA

ISBN-13 (pbk): 978-1-4842-2942-2
<https://doi.org/10.1007/978-1-4842-2943-9>

ISBN-13 (electronic): 978-1-4842-2943-9

Library of Congress Control Number: 2017959876

Copyright © 2017 by Laurence Moroney

This work is subject to copyright. All rights are reserved by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.

Trademarked names, logos, and images may appear in this book. Rather than use a trademark symbol with every occurrence of a trademarked name, logo, or image we use the names, logos, and images only in an editorial fashion and to the benefit of the trademark owner, with no intention of infringement of the trademark.

The use in this publication of trade names, trademarks, service marks, and similar terms, even if they are not identified as such, is not to be taken as an expression of opinion as to whether or not they are subject to proprietary rights.

While the advice and information in this book are believed to be true and accurate at the date of publication, neither the authors nor the editors nor the publisher can accept any legal responsibility for any errors or omissions that may be made. The publisher makes no warranty, express or implied, with respect to the material contained herein.

Cover image by Freepik (www.freepik.com)

Managing Director: Welmoed Spahr
Editorial Director: Todd Green
Acquisitions Editor: Steve Anglin
Development Editor: Matthew Moodie
Technical Reviewer: Val Okafor
Coordinating Editor: Mark Powers
Copy Editor: Karen Jameson

Distributed to the book trade worldwide by Springer Science+Business Media New York, 233 Spring Street, 6th Floor, New York, NY 10013. Phone 1-800-SPRINGER, fax (201) 348-4505, e-mail orders-ny@springer-sbm.com, or visit www.springeronline.com. Apress Media, LLC is a California LLC and the sole member (owner) is Springer Science + Business Media Finance Inc (SSBM Finance Inc). SSBM Finance Inc is a **Delaware** corporation.

For information on translations, please e-mail rights@apress.com, or visit <http://www.apress.com/rights-permissions>.

Apress titles may be purchased in bulk for academic, corporate, or promotional use. eBook versions and licenses are also available for most titles. For more information, reference our Print and eBook Bulk Sales web page at <http://www.apress.com/bulk-sales>.

Any source code or other supplementary material referenced by the author in this book is available to readers on GitHub via the book's product page, located at www.apress.com/9781484229422. For more detailed information, please visit <http://www.apress.com/source-code>.

Printed on acid-free paper

Contents at a Glance

About the Author	xi
About the Technical Reviewer	xiii
■ Chapter 1: An Introduction to Firebase	1
■ Chapter 2: Using Authentication in Firebase	25
■ Chapter 3: The Firebase Realtime Database.....	51
■ Chapter 4: Cloud Storage for Firebase.....	73
■ Chapter 5: Using Firebase Hosting	93
■ Chapter 6: Using Test Lab for Android	107
■ Chapter 7: Understanding Crashes.....	127
■ Chapter 8: Cloud Functions for Firebase	139
■ Chapter 9: Firebase Cloud Messaging	163
■ Chapter 10: Firebase App Indexing	189
■ Chapter 11: Remote Configuration	203
■ Chapter 12: Dynamic Links and Invites	215
■ Chapter 13: Using AdMob	233
■ Chapter 14: Google Analytics for Firebase	251
Index.....	271

Contents

About the Author	xi
About the Technical Reviewer	xiii
■ Chapter 1: An Introduction to Firebase	1
Firebase: An Overview	2
The Develop Technologies	3
The Grow Technologies	4
Earning with AdMob	5
Google Analytics for Firebase	6
Getting Started with Firebase on Android	6
Install Android Studio	6
Create Your First Firebase Application	9
Summary	24
Reference	24
■ Chapter 2: Using Authentication in Firebase	25
FirebaseUI	26
Email/Password Sign-In	28
Create and Configure the App	28
Code the Main Activity	32
Create and Code the Signed-In Activity	35

Explore the Sign-In Flows.....	37
Adding Sign Out.....	40
Resetting the Password.....	42
Adding Smart Lock for Passwords	44
Using Federated Identity Providers	47
Adding Sign-In with Google.....	47
Summary.....	50
■ Chapter 3: The Firebase Realtime Database.....	51
Getting Started	51
Write Your First Data.....	55
Read Your First Data	58
Using Java to Structure Data.....	61
Write Structured Data with a Stock Portfolio.....	61
Reading Structured Data	67
Updating Data	69
Deleting Data.....	70
Summary	71
■ Chapter 4: Cloud Storage for Firebase.....	73
Building an App with Cloud Storage for Firebase.....	73
Enable Anonymous Authentication	74
Set Storage Rules.....	75
Editing the Layout.....	76
Editing the Code for Anonymous Sign-In.....	77
Taking a Photo with the Onboard Camera	78
Uploading the Image to Cloud Storage	79
Running the App	81
Building an App to Download from Cloud Storage	84
Edit the Layout File.....	84
Coding the App	85

Reading Metadata	90
Update the Layout File	90
Next Steps	92
Summary	92
■ Chapter 5: Using Firebase Hosting	93
The Firebase CLI	93
Installing the Firebase CLI	93
Creating a Site for Firebase Hosting	96
Using the Firebase Console to Manage Your Site	98
A Real-World Example: Handling Zoomable Images	99
Preparing the Image	99
Creating the Page	100
Mapping a Domain Name to Your Site	103
Summary	106
■ Chapter 6: Using Test Lab for Android	107
Getting the App	107
Enabling Authentication	108
Enabling Database Rules	109
Using Friendly Chat	109
First Steps: Testing Your APK	110
Creating Custom Tests	114
Testing from Android Studio	118
Recording and Scripting a Test with Espresso	122
Summary	126
■ Chapter 7: Understanding Crashes	127
Getting Started with Crashlytics	127
Installing Crashlytics	128
Using Firebase Crash Reporting	136
Summary	138

■ Chapter 8: Cloud Functions for Firebase	139
Building a Test App	139
The Firebase CLI and Functions	143
Triggers	148
Realtime Database Triggers	148
Storage Triggers	152
Authentication Triggers	154
Using Http Triggers	156
Http Triggers with Hosting	159
Other Triggers	161
Summary	161
■ Chapter 9: Firebase Cloud Messaging	163
Building an App to Receive Notifications	163
Receiving Notifications in the Foreground	166
Implementing the Notification Activity	168
Implementing the sendNotification Method	170
Understanding Topics	174
Sending Messages Using Analytics	177
Building a Custom App Server for FCM	177
Creating the Database	179
Updating the App	184
Next Steps	187
Summary	188
■ Chapter 10: Firebase App Indexing	189
Public Content Indexing	189
Create an App That Receives Incoming Links	190
Connect a Site to the App	194
Personal Content Indexing	194
Going Further	201
Summary	201

■ Chapter 11: Remote Configuration	203
Building an App for Remote Config	204
Using Default Values	207
Using Conditions with Remote Config	209
Remote Config Life Cycle	212
Summary	213
■ Chapter 12: Dynamic Links and Invites	215
Dynamic Links	215
Editing the App	219
Creating a Dynamic Link Within the App	223
Dynamic Links and Analytics	227
Using Firebase Invites	228
Summary	231
■ Chapter 13: Using AdMob	233
Getting Started	233
Signing Up for AdMob	235
Testing with a Device	240
Using Interstitial Ads	240
Using a Rewarded Video Ad	244
Connecting AdMob with Firebase	247
Summary	250
■ Chapter 14: Google Analytics for Firebase	251
Automatically Gathered Analytics	251
Exploring the Analytics Console	253
Creating Audiences	262
Coding for Analytics	263
Logging Events	263
Logging User Properties	269
Summary	270
Index	271

About the Author



Laurence Moroney is Firebase Developer Advocate at Google. He is also a top-selling author. He is host of “Coffee with a Googler” on YouTube! He has over 10 years in software development and architecture, specializing in interoperability, security, and performance in such diverse industries as casinos, jails, the U.S. Border Patrol, airports, professional soccer teams, and financial services. He has written several books on computing, including some on Web Services Security, ASP.NET, and Java/.NET interoperability, as well as dozens of articles on various technology issues. He lives in Sammamish, Washington, with his wife, Rebecca; and children, Claudia and Christopher.

About the Technical Reviewer



Val Okafor is a software architect with expertise in Android development and resides in sunny San Diego of California, USA. He has over 12 years of industry experience and has worked for corporations such as Sony Electronics, The Home Depot, San Diego County, and American Council on Exercise. Val earned his BSc in IT from National University, San Diego; and Masters in Software Engineering from Regis University, Colorado. He is the creator and principal engineer of Pronto line of mobile apps including Pronto Diary, Pronto Invoice, and Pronto Quotes.

His passion for software development goes beyond his skill and training; he also enjoys sharing his knowledge with other developers. He has taught Android development to over 5,000 students through Udemy, and his blog valokafor.com is considered an essential reading for Android developers. Val was also recently named among the first cohort of Realm MVP program because of his active participation in the Realm database community.

Chapter 1

An Introduction to Firebase

In a survey conducted in 2016,¹ it was found that 52% of developers fall below what is called the *poverty* line, \$500 or less per month, in mobile revenue. Only 18% of developers were in the high earning category – \$25k or more per month. Of those in the high earners bracket, 88% targeted three platforms: Android, iOS, and the Web. Thus, it would be logical that in order to emulate the most successful developers, you would want to target these platforms. However, doing so can be difficult when it comes to skillsets – managing separate code bases for complex tasks such as database management, identity, messaging, and other common attributes of apps will greatly increase the time and effort you need to invest to build and launch your app.

Additionally, marketing and growing an app can be an onerous task. One cannot simply put an app into the Play Store or App Store and expect the downloads to keep coming. In an increasingly crowded marketplace, continued investment and continued effort to grow the app are necessary to prevent your app from being lost in the crowd.

Finally, earning from your hard work is also important. Otherwise you wouldn't be spending the time and effort in creating the app to begin with! There are a number of ways that you can earn from an app – first, of course, is to charge for it. But increasingly, developers are going with a no-cost option, where users can get your app without paying for it, and are then monetized by in-app content, advertising, or both.

With these three needs in mind, Google released Firebase in the summer of 2016. Its goal is to provide the tools and infrastructure that you need to build great apps, grow a successful business, and earn from your hard work.

It's not a *replacement* for your existing APIs for building Android, iOS, or Web apps. It's an *enhancement*, giving you common services that you might need – such as a database back end, secure authentication, messaging, and more. This saves you the need to build them yourself, allowing you to focus on what makes your app distinct. Additionally, it has technologies that you can put into your app and site that will help you grow your business through referrals, linking, and more. It has an easy-to-use Advertising API that you can drop into your app to start earning, and importantly, the whole platform is tied together by analytics.

Let’s take a look at the full Firebase stack. In this chapter we’ll take a brief look at each of the technologies, and later we’ll do a deeper dive into each one. In this book you’ll be learning how to use each of these in an Android app. The principles for iOS are very similar, but all the code here will be for Android.

Firebase: An Overview

Firebase is built on three pillars: Develop, Grow, and Earn, and represented in Figure 1-1:

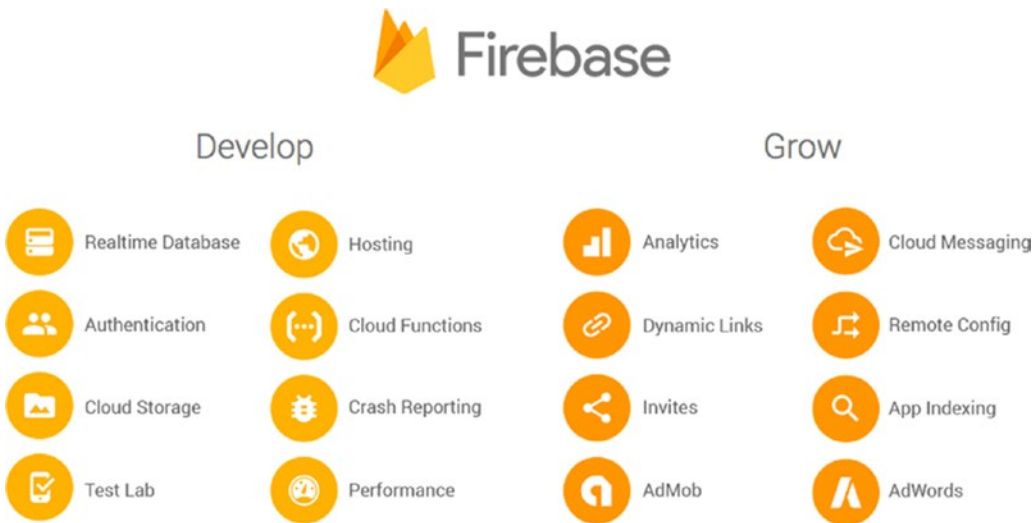


Figure 1-1. The Firebase Pillars

Each of these technologies is tied together using Analytics. It’s important to note that you can choose which of these technologies that you want to use in your app or site. You aren’t required to use them all.

Many of the technologies are available at no cost. These include Analytics, App Indexing, Authentication (with the exception of Phone Auth), Cloud Messaging, Crash Reporting, Dynamic Links, Invites, Notifications, and Remote config.

For all the others, there is a free tier that will work for testing and with reasonable limits for smaller apps. For example, for the Realtime Database, the free tier will allow you to store 1Gb of Data and have 100 simultaneous connections. Similarly for Cloud Functions for Firebase, the free tier allows you 125,000 invocations per month. Full details on pricing are available at <https://firebase.google.com/pricing/>.

Let’s look at the columns in turn.

The Develop Technologies

Firebase has eight technologies that are designed to enhance your app development experience. These are listed on the left of the diagram above.

Firestore Authentication

When your app needs to know the identity of the user in order to provide distinctive data to that user, a form of secure sign-in is necessary. Building and maintaining sign-in infrastructure is a difficult and expensive proposition. From a user perspective, giving credentials and personal data to an app can also be a user experience bump – potentially causing them to reject using it. Thus, apps that allow users to sign in with known credentials, such as those provided by Google, Facebook, and Twitter, are becoming more popular. With these issues and trends in mind, Firestore Authentication has been built to give you an easy API that will allow you to use sign-in from federated providers, a simple email/password scheme, or integrate with any existing authentication back ends you own. It integrates with Firestore services such as the Realtime database so you can control who accesses what data. You'll learn how to build for Firestore Authentication in Chapter 2.

The Realtime Database

This is a cloud-hosted NoSQL-based database. It provides syncing across connected devices and is available when there is no network connectivity through a local cache. It is an event-driven database that works very differently from traditional SQL databases. There's no server-side code and database access tiers; all coding is done in the client. Whenever data changes in the database, events are fired in the client code, and you can then handle and update the state of your user interface in response. It contains an expression-based rules language, called the Firestore Realtime Database Security Rules, which define how data is structured and which users have rights to that data. Its ultimate design is to be *responsive*, allowing you to build a realtime experience that can serve users at high scale. As such it is a little different to program against than a traditional database. It's also very simple – and we'll cover it in Chapter 3.

Cloud Storage for Firestore

In addition to storing data, it's a common requirement that apps store files such as photos or videos. Building and managing an infrastructure to handle this – particularly one that deals with large files such as videos – can be a difficult and expensive proposition. Cloud Storage for Firestore makes this easier by providing a simple API that is backed up by Google Cloud Storage. In Chapter 4, you'll look at this API, and see how you can easily handle uploading and downloading of files, including what to do to handle communications failures and resume where you left off.

Firestore Hosting

All apps need an associated web site for their Play Store or App Store listing. If you're using Firestore, you'll get hosting space that you can use to host static assets such as HTML, CSS, JavaScript, or images. Using JavaScript frameworks you can build some pretty sophisticated sites. In Chapter 5 you'll learn about Firestore Hosting, and you'll step through building a site with it.

Firestore Test Lab

A common complaint for App Developers, particularly those that use Android, is that it is very difficult to get access to all types of devices that your end users might need. In many cases, the device isn't even available in your country. With Firestore Test Lab, you can take advantage of devices that are hosted by Google in a Test Center for you! In Chapter 6 you'll learn about tests that have already been written for you by Google (aka Robo Tests) as well as how to build your own tests. You'll see the results in the form of screenshots and crash reports.

Firestore Crash Reporting

The number 1 reason for bad reviews in the App Store and Play Store is unexpected crashes. Figuring out why an app crashed, and fixing it quickly is paramount, but a very difficult process if the app is run on a device that you don't know, in a country or city that's far away! Crash Reporting helps with this by providing a Stack Trace of all crashes in the Firestore Console. You can use these to figure out the cause of the crash, and roll out a fix quickly. You might even be able to use Firestore Cloud Messaging to let your users know of the update, so they can avoid the crash in the Future! You'll take a look at this technology in Chapter 7.

Cloud Functions for Firestore

While the Firestore architecture is primarily geared toward code running in your mobile app or web site front end, there are often circumstances where you need logic to execute on the back end. Cloud Functions for Firestore gives you the facility to write code that responds to events on Firestore elements. For example, changes in the Realtime Database, Authentication, or other elements can be used to trigger a function. You'll see more about Cloud Functions and learn how to use them in Chapter 8.

The Grow Technologies

In addition to providing services that help you build apps, Firestore has a number of technologies that can be used to help you systematically grow your app.

Firestore Cloud Messaging

In Chapter 9 you'll learn about Firestore Cloud Messaging and Firestore Notifications. These allow you to reliably deliver messages at no cost. Over 98% of connected devices receive these messages in less than 500ms. Messages can be delivered in a variety of ways – driven by analytics to pick audiences, or using topics or other methods.

Firestore Remote Config

Remote Config is a cloud service that provides server-side variables that can be driven by analytics, allowing you to change the behavior and/or appearance of your app without requiring an app update. For example, if you are building an e-commerce app, and you want to periodically provide discounts to your users, you could have a remote config variable that contains the value of the discount. Traditionally it could be 0%, but at certain times of the year you could set it to 10%. But then you could go further – what if you want to give a bigger discount to people in a country that you want to grow in – you can set the value based on analytics to give them, say 20%. In Chapter 10, you'll learn how to do this.

App Indexing

A common scenario is that a user installs your app, but then it gets lost in their universe of applications. But an everyday scenario is that they search on their phone. Logically, if they are searching for content that's already in your app, and they previously made the decision to install that app, then the app should be surfaced, and the content should be opened in that app directly from search. App Indexing allows you to do this, and in Chapter 11, you'll see how to link your site and app so that Google Search knows to make the connection!

App Invites and Dynamic Links

Friends help friends discover great apps. App Invites is a technology that, should you implement it in your app, will make the process a lot simpler. It allows your user to choose who to send the invite to from an intelligently sorted list. You'll learn how to do this in Chapter 12, including how to use Dynamic Links technology that underpins it.

Earning with AdMob

Chapter 13 will introduce you to how you can use AdMob in your app. You'll learn about how to use banner ads, interstitial ads, video, or native ads within your app. This will require an AdMob account, and you'll be stepped through this.

Google Analytics for Firebase

At the heart of Firebase is analytics, a solution that's free to use and very comprehensive. It has a number of common analytics that it will gather for you without writing any code, as well as the ability to define custom analytics that will be gathered for you. It helps you understand your audience well, which can be used to continually grow your app, as well as driving technologies such as Firebase Remote Config, and Cloud Messaging!

Getting Started with Firebase on Android

For this book, the main development tool you will use is Android Studio. This is available at developer.android.com/studio. For the purposes of writing this book, I used version 2.3.1 for Mac, so if you're using a later version, some of the screenshots may differ, but the principles should be the same.

Install Android Studio

You can download Android Studio from <https://developer.android.com/studio>. When you visit the page, you'll see something like Figure 1-2:

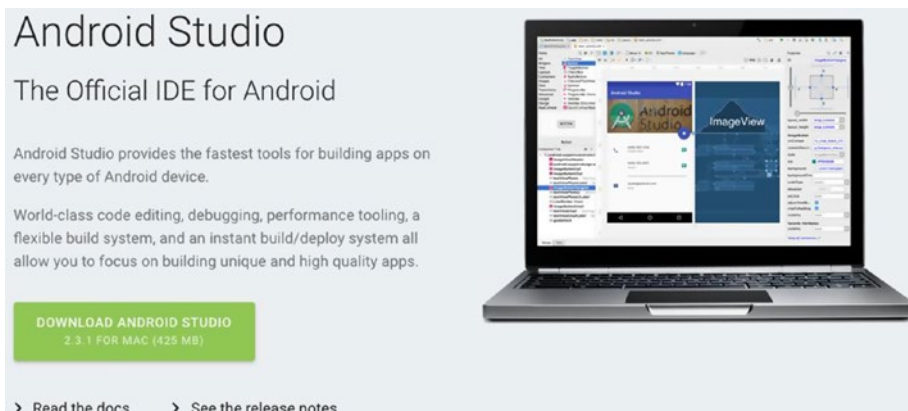


Figure 1-2. Android Studio Download Site

Click the button to download and install Android Studio. Depending on your platform, the experience may differ.

Once you've gone through the install process, launch Android Studio for the first time. You'll likely see a welcome screen like that in Figure 1-3.

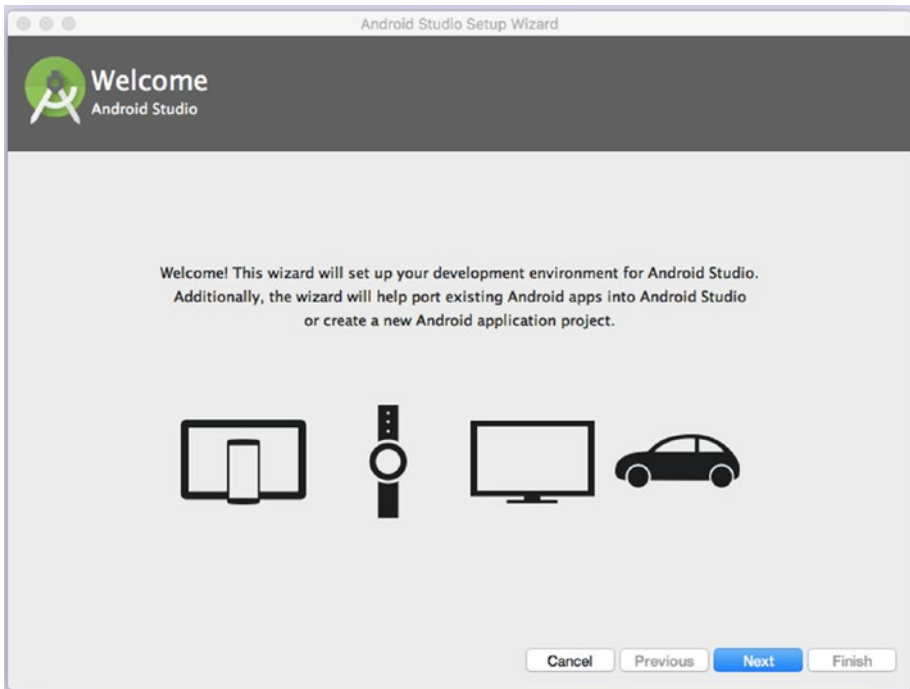


Figure 1-3. *Welcome to Android Studio*

Press 'Next' and you'll be asked for the Install Type. You can choose 'Standard' or 'Custom'. Choose 'Standard' and press 'Next'.

You'll then be given a number of settings that Android Studio needs to download and install. Click 'Finish' to let it do them. Once they're done, press 'Finish' and you'll be taken to the next step. Note that you may be asked for your password on some stages – such as installing HAXM (which speeds up the emulator). It's perfectly okay to do so.

Upon finishing you'll see the screen in Figure 1-4, letting you know that you're ready to go in building a new project.

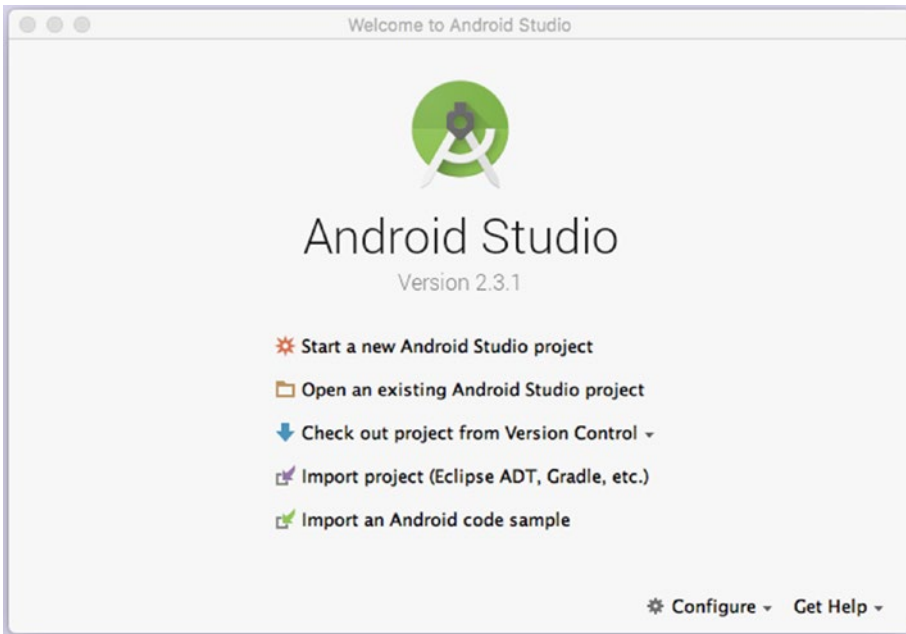


Figure 1-4. Starting Android Studio

You'll still need to install the Firebase tools, and you'll see that in the next section.

Create Your First Firebase Application

From the screen in Figure 1-4, select ‘Start a new Android Studio project’. You’ll be asked to give the project a name and a domain. Give it the name `FirebaseCh1`, but choose a company domain name that’s unique to yourself. In mine I called it `laurencemoroney.com`, but you should choose something else! This will determine the package name for your app. See Figure 1-5.

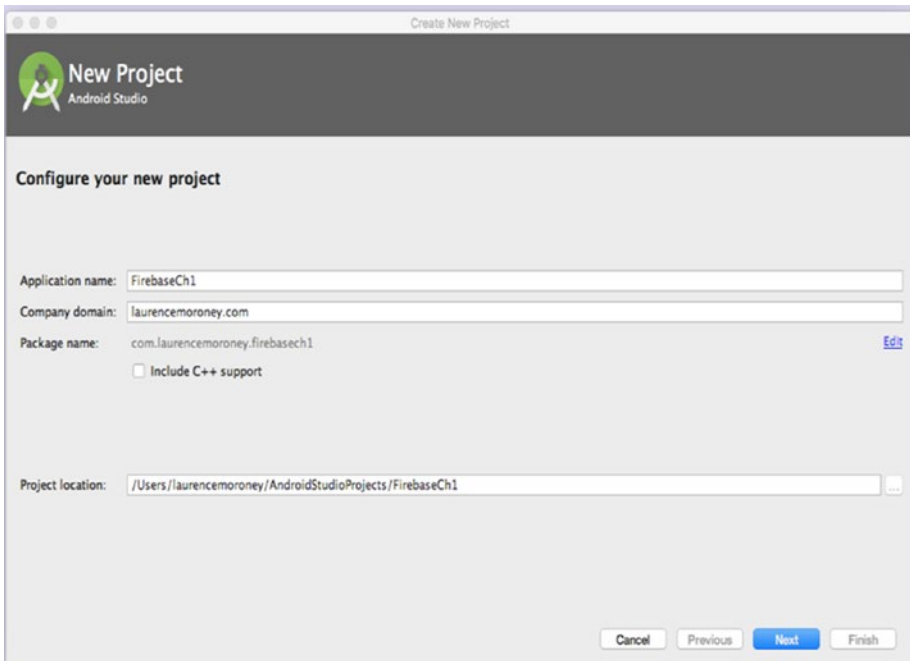


Figure 1-5. Creating a new Project

Click Next and you’ll be asked about the form factors to run on. Just accept the defaults, and click Next. The app will be created, and once you see ‘Done’, click Next. You’ll be asked to add an activity to your app. For simplicity, pick the ‘Empty Activity’. See Figure 1-6.

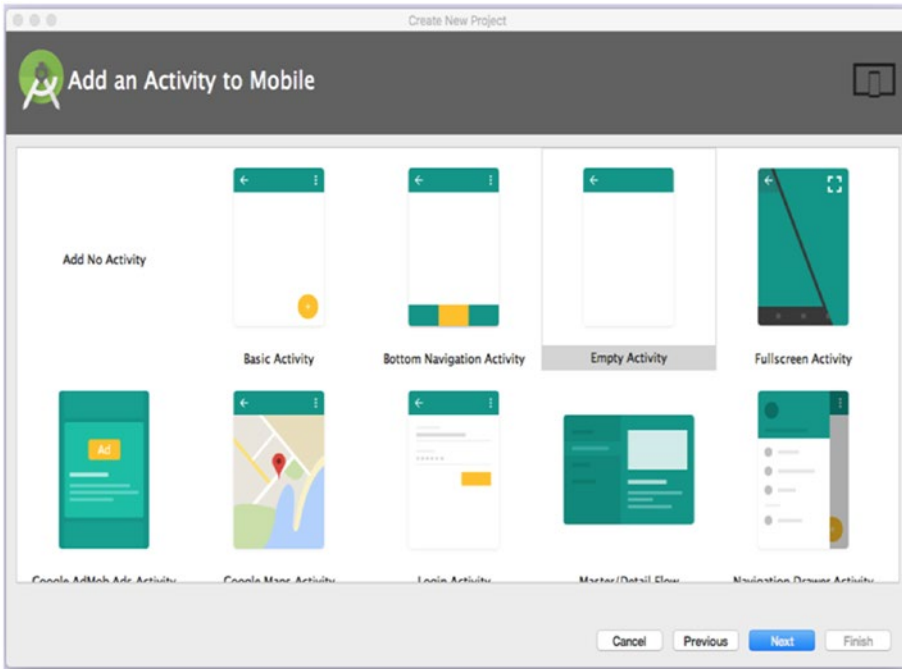


Figure 1-6. Adding an Activity to your App

You'll be asked to give this activity a name. Just take the defaults, and press 'Finish'. The project will be created and the Android Studio Integrated Development Environment (IDE) will start up. You now have a boilerplate Android app, which doesn't do very much.

Let's start by adding Firebase to it. You'll create some code that writes data to the realtime database, and which also reads it back!

In Android Studio's Tools menu, you should see a 'Firebase' option. See Figure 1-7.

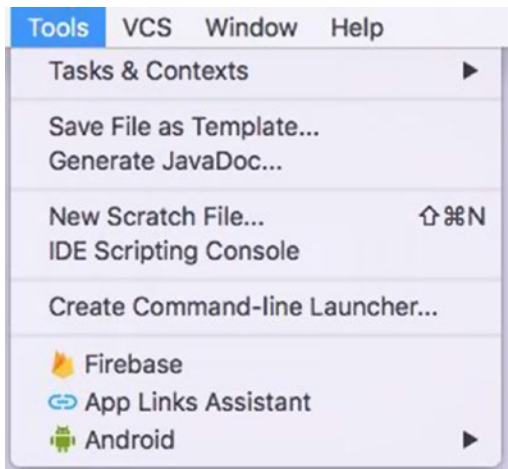


Figure 1-7. Android Studio Tools Menu

Select 'Firebase' and the Firebase Assistant will open on the right-hand side of Android Studio. See Figure 1-8. This gives you a number of handy shortcuts to creating Firebase projects, handling dependencies, and even some boilerplate code!

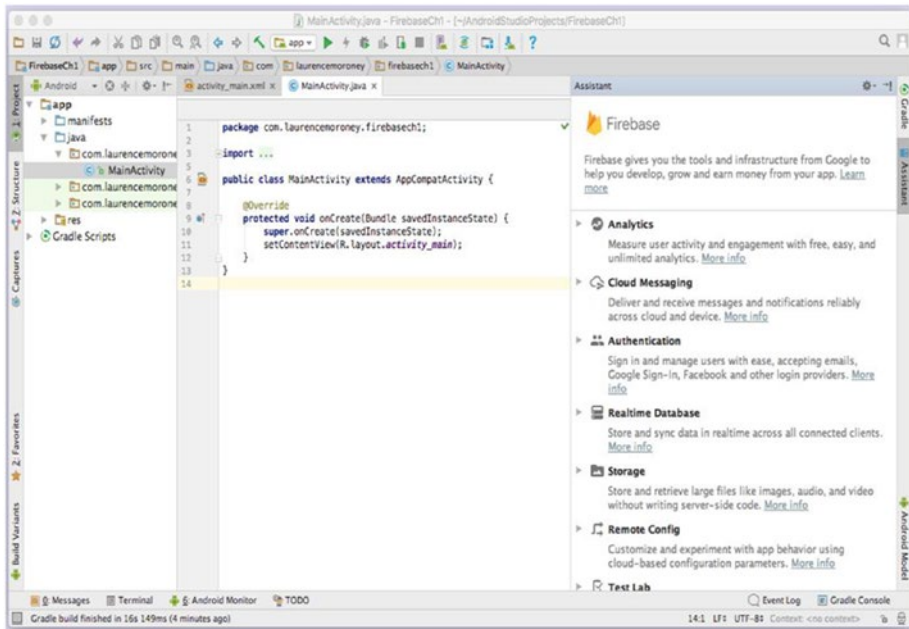


Figure 1-8. Android Studio with the Firebase Assistant

You'll be using this assistant extensively as you work through this book, so let's look at a simple example to get started. You aren't limited to using the assistant, of course, and can add a Firebase project manually to your Android application by downloading the Google-Services.json file from the Firebase Console, and setting up the required libraries in your build.gradle. For simplicity, I'll use the assistant throughout this book.

Find the section that says 'Realtime Database' and expand it. Your screen should look something like Figure 1-9.

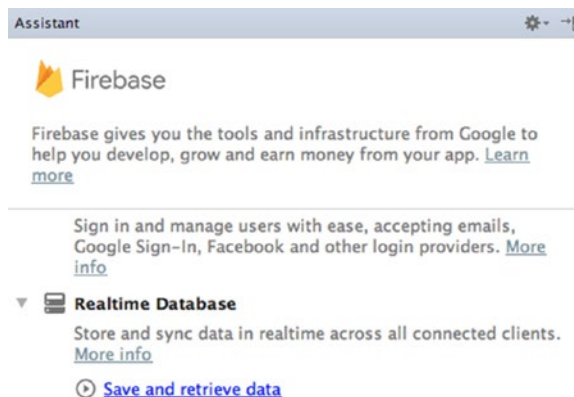


Figure 1-9. Using the Realtime Database in Firebase

Click the ‘Save and Retrieve data’ link. The assistant will change to give you a step-by-step process to follow to connect to the Realtime database. See Figure 1-10.

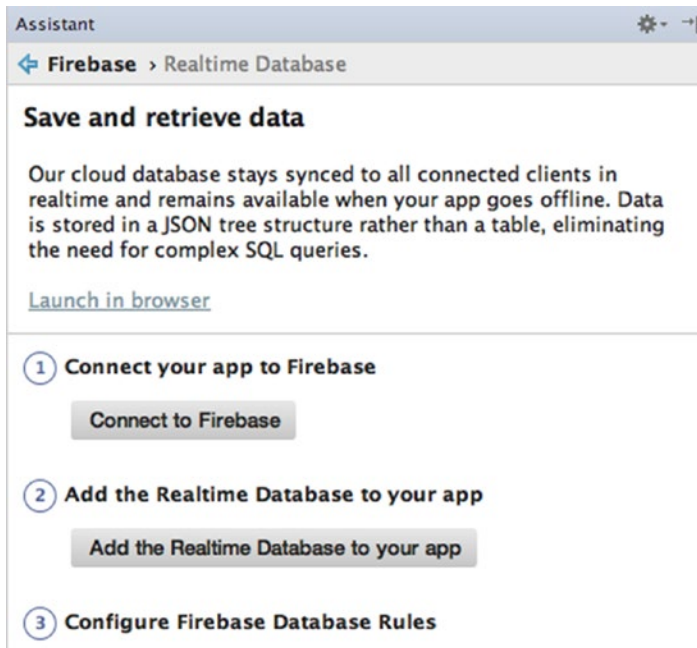


Figure 1-10. Realtime Database Steps

Let’s go through these steps.

Step 1. Connect to Firebase

Step 1 connects your Android app to Firebase. As Firebase is a back end as a service, it needs to run in the cloud, and be administered through a cloud-based console. You create projects in this console and associate them with apps built in Android Studio, XCode, or elsewhere. The assistant automates this process for you.

Press the ‘Connect to Firebase’ button, and a browser page will open asking you to sign in. This will be your sign-in to the Firebase Console, so it asks for all the relevant permissions. When you’re done, you’ll see a success message in the browser. Close this, and Android Studio will give you the dialog in Figure 1-11 that you’ll use to connect to Firebase. Here, you can create a new Firebase project to connect to, or if you have existing ones, you can connect this Android app to them. Go ahead and select ‘Create New Firebase project’, and you’ll see it defaults to your app name. See Figure 1-11.

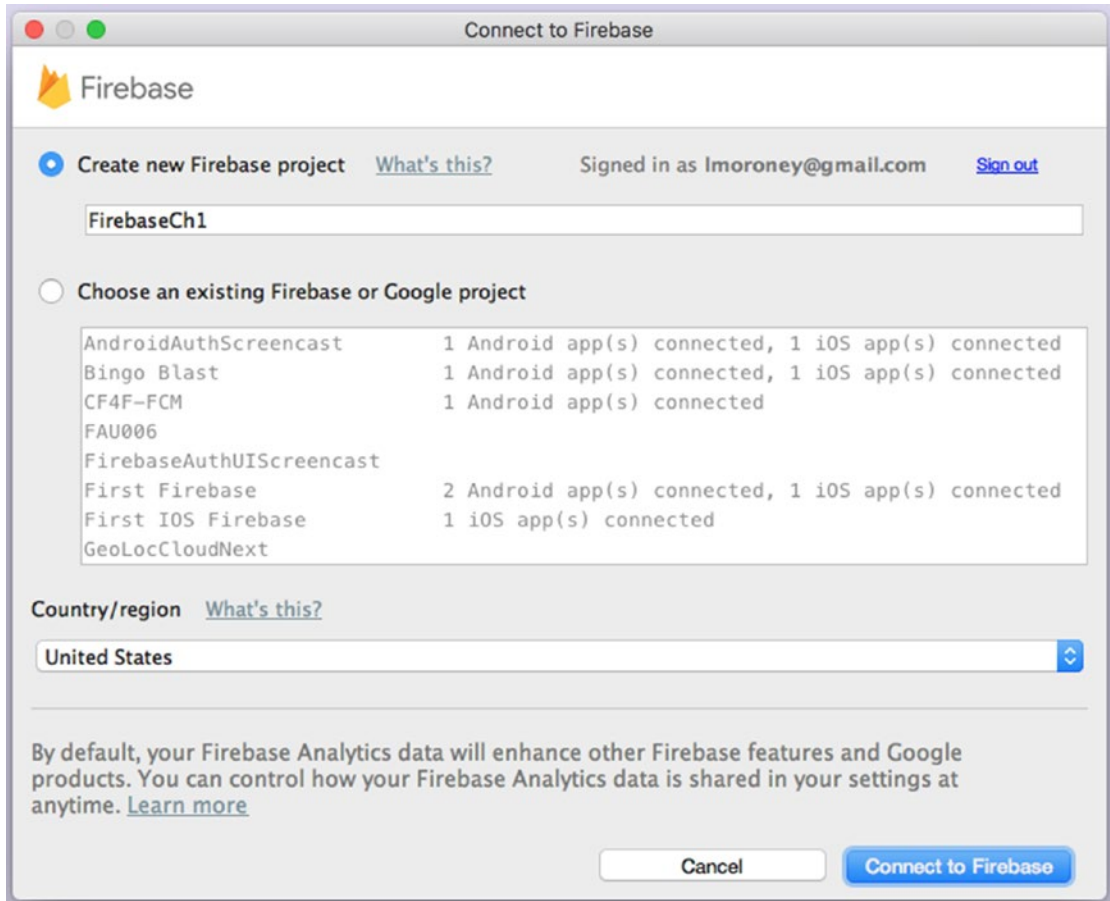


Figure 1-11. Connect your App to Firebase

When you click the 'Connect to Firebase' button, Android Studio will create the project on the Firebase Console, as well as add all the required dependencies to your Android app. This will take a few moments, but when you're done, you'll see that the status in the assistant has changed to 'Connected'. See Figure 1-12.

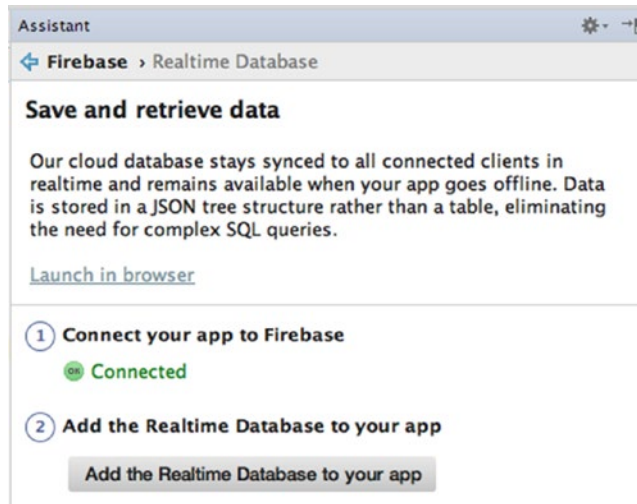


Figure 1-12. Updated assistant to show app is connected to Firebase

You're now ready to proceed to Step 2 – Adding the Realtime Database to your App.

Step 2. Add the Realtime Database

Click the 'Add the Realtime Database to your app' button, and you'll see a dialog mentioning that some changes will be made to your build.gradle files. Go ahead and accept these changes, and the configuration will be updated and synced. It may take a few moments.

Once it's done, you'll see that your build.gradle files have been updated. Here's an example:

apply **plugin: 'com.android.application'**

```
android {
    compileSdkVersion 25
    buildToolsVersion "25.0.3"
    defaultConfig {
        applicationId "com.laurencemoroney.firebaseioch1"
        minSdkVersion 15
        targetSdkVersion 25
        versionCode 1
        versionName "1.0"
        testInstrumentationRunner "android.support.test.runner.AndroidJUnitRunner"
    }
    buildTypes {
        release {
            minifyEnabled false
            proguardFiles getDefaultProguardFile('proguard-android.txt'), 'proguard-rules.pro'
        }
    }
}
```

```
dependencies {
    compile fileTree(dir: 'libs', include: ['*.jar'])
    androidTestCompile('com.android.support.test.espresso:espresso-core:2.2.2', {
        exclude group: 'com.android.support', module: 'support-annotations'
    })
    compile 'com.android.support:appcompat-v7:25.3.1'
    compile 'com.android.support.constraint:constraint-layout:1.0.2'
    compile 'com.google.firebase:firebase-database:10.0.1'
    testCompile 'junit:junit:4.12'
}
```

apply plugin: 'com.google.gms.google-services'

Note the dependency for 'com.google.firebase:firebase-database:10.0.1' that is the libraries for the Firebase database and the 'com.google.gms.google-services' plug-in also being applied.

You'll also see that the assistant has updated Step 2 to be complete. See Figure 1-13.

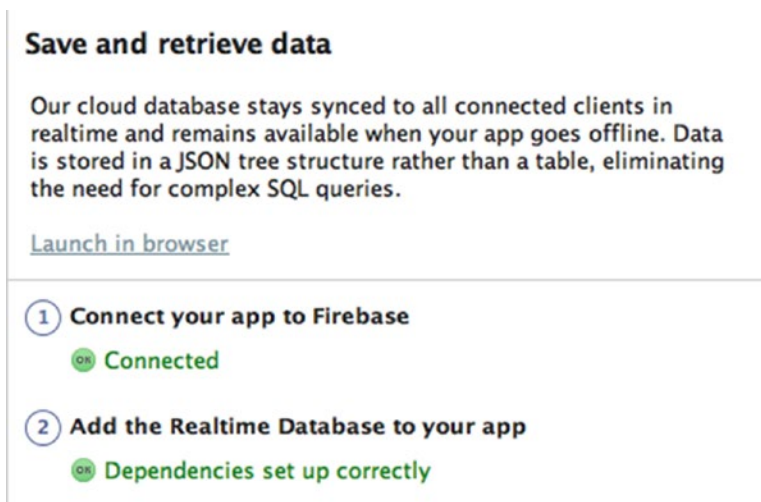


Figure 1-13. Step 2 is complete

This also gives you the google-services.json file that is autogenerated by the Firebase Console. This file is used to define a number of parameters that Firebase and the underlying Google Services need to operate on Android. You can learn more about the file and its structure here: <https://developers.google.com/android/guides/google-services-plugin>.

Next up you'll configure the data access rules using the Firebase Console.

Step 3. Configure Data Access Rules

The next step in the assistant asks you to set up your data access rules. You'll go into these in more detail in Chapter 2, but for now you'll just set up your data for public access by following the link in the text. This will take you to the documentation web site showing you how to do this. Note that there are several tabs on the rules section. Be sure to pick the 'Public' tab so you can see what public rules look like. They simply allow anybody to read and write data on your app. See Figure 1-14.

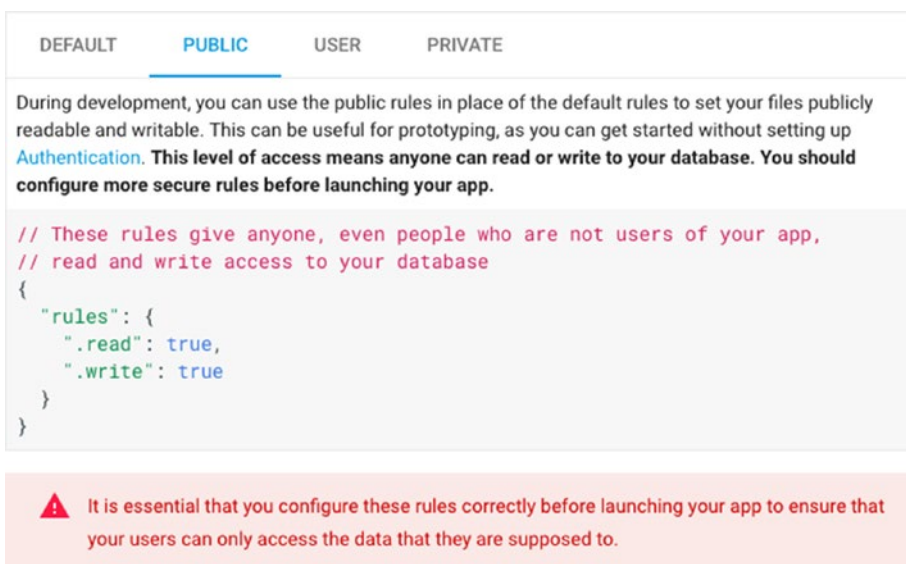


Figure 1-14. Configuring Data Access Rules

These settings are used in the Firebase Console. On the top right of the browser window you'll see a link that says 'Go to Console'. Select that, or just visit <https://console.firebase.google.com>. This will take you directly to the console where you can edit the project. It should look something like Figure 1-15. In this case, I have a number of projects I've been working on, so don't worry if yours looks different!

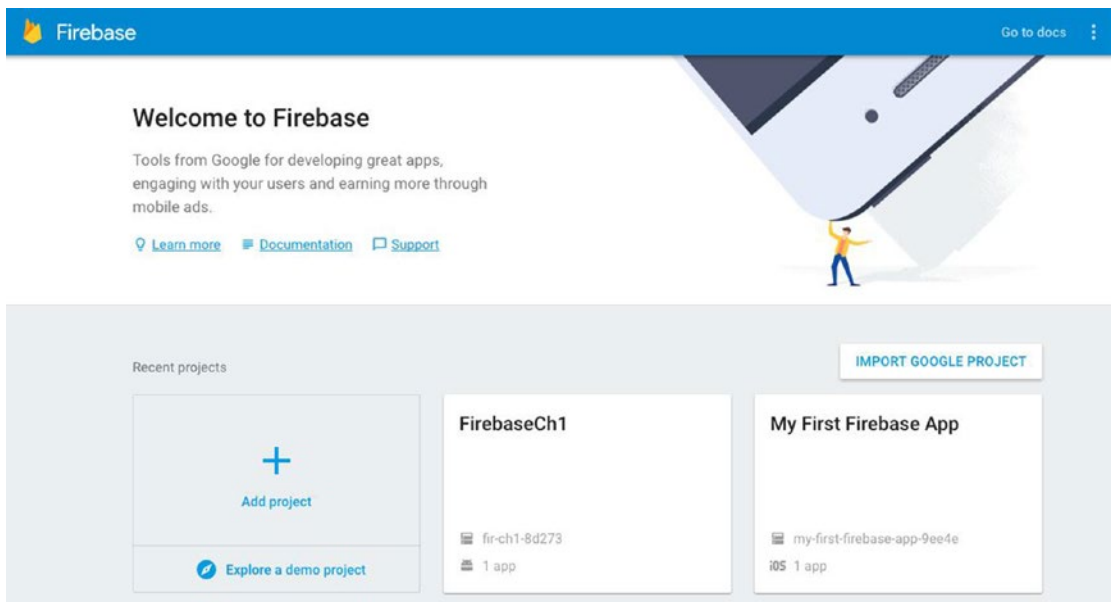


Figure 1-15. The Firebase Console

If you called your project `FirebaseCh1`, you'll see its tile here. Otherwise, choose the tile for whatever you called the app. Clicking on it will take you to the console for the app. You'll see tiles for the mobile apps in this project, as well as a list of links on the left-hand side for the elements of Firebase. See Figure 1-16.

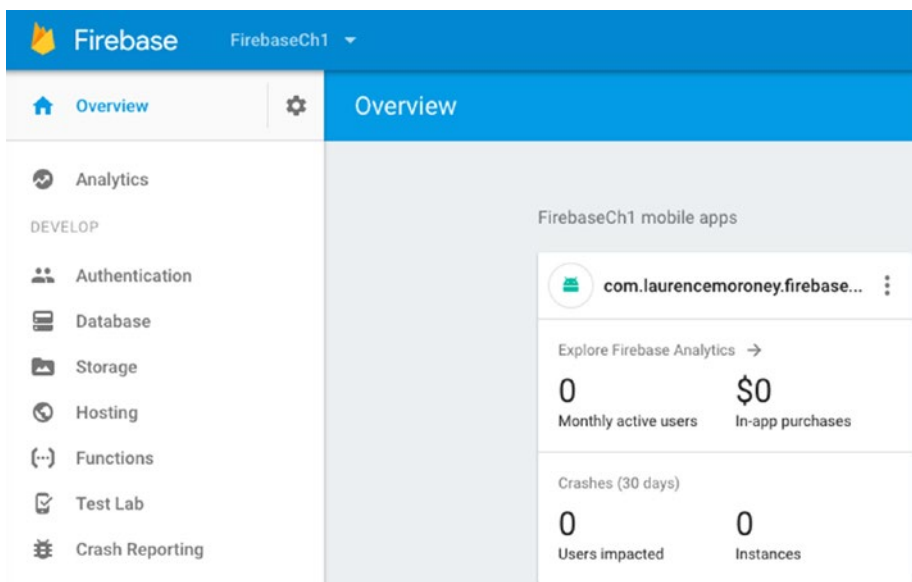


Figure 1-16. The Firebase Console for this project

Select 'Database' on the left-hand side, and you'll get taken to the Database settings page. At the top of this, you'll see tabs for 'Data', 'Rules', 'Backups', and 'Usage'. Select 'Rules' and you'll see something like Figure 1-17.

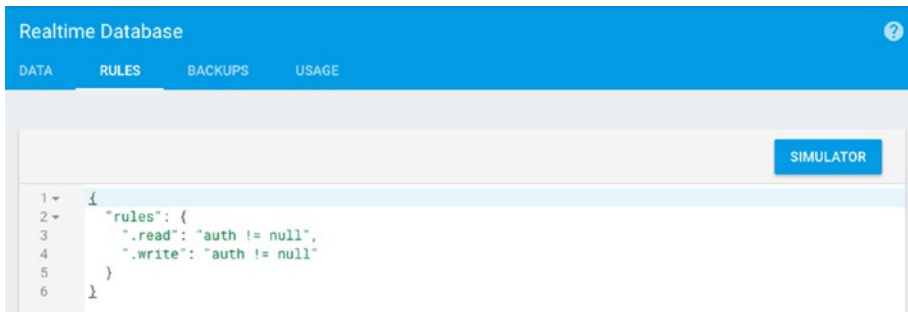


Figure 1-17. Setting the Database Rules

Edit the rules to match the public settings you saw earlier – in this case you set `.read` and `.write` to `true`, meaning everybody can read and write this data. Remember that you should never architect a public app in this way, or anybody will be able to do whatever they want to your data! But for learning and testing, it's okay for now.

Once you've done this, you'll see a 'Publish' button appears. See Figure 1-18.

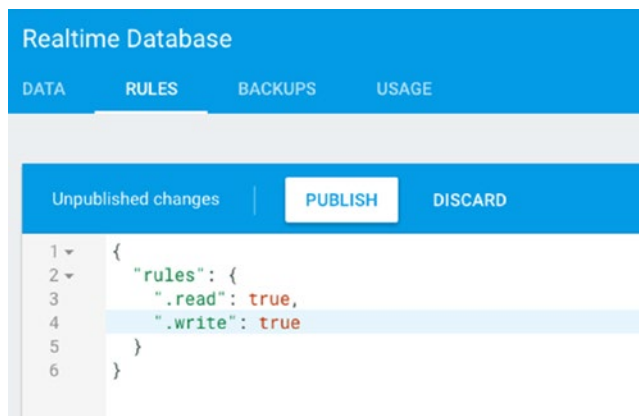


Figure 1-18. Editing and Publishing the rules

Press 'Publish' and the rules will be published to the database. You've now finished Step 3!